



ORACLE
NetSuite

Financial Institution Connectivity Plug-in

2025.2

December 17, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Financial Institution Connectivity Plug-in Interface Overview	1
Financial Institution Connectivity Workflows	2
Default Financial Institution Connectivity Plug-in Implementations	4
Capabilities of Using a Financial Institution Connectivity Plug-in	4
Limitations of Using a Financial Institution Connectivity Plug-in	5
Developing a Financial Institution Connectivity Plug-in	5
Enabling Features for a Financial Institution Connectivity Plug-in	5
Creating a Financial Institution Connectivity Plug-in Script File	5
Administering a Financial Institution Connectivity Plug-in	7
Enabling Features for a Financial Institution Connectivity Plug-in	7
Adding the Financial Institution Connectivity Plug-in Implementation	7
Configuration for a Financial Institution Connectivity Plug-in	8
Conditions for Running a Financial Institution Connectivity Plug-in	8
Conditions for Testing a Financial Institution Connectivity Plug-in	9
Financial Institution Connectivity Plug-in Interface Definition	11
getConfigurationIFrameUrl()	11
pluginConfiguration	11
configurationIFrameUrl	12
getAccounts()	13
pluginConfiguration	13
addAccount()	14
getTransactionData()	16
pluginConfiguration	17
accountRequestsJSON	18
addDataChunk()	18
returnAccountRequestsJSON()	18
Error Propagation	19

Financial Institution Connectivity Plug-in Interface Overview



Important: The Financial Institution Connectivity Plug-in interface uses SuiteScript 2.0. Your entry point scripts and supporting library scripts must also use SuiteScript 2.0. For information, see the help topics [SuiteScript 2.x Script Creation Process](#) and [SuiteScript 2.x](#).

Use a Financial Institution Connectivity Plug-in to directly connect to financial institutions and automatically receive inbound transmissions through NetSuite, daily. Inbound transmission enables you to automate cash balance reporting and schedule account statement downloads.

To create a Financial Institution Connectivity Plug-in, you need to create the plug-in implementation script file, and then upload the script file and other utility files as required. For details, see [Creating a Financial Institution Connectivity Plug-in Script File](#).

You can define alternative implementations that suit your needs by customizing the core plug-in logic. To accommodate this process, NetSuite provides an interface definition that describes the name, parameters, and return type of each function defined in the plug-in's interface. For details, see [Financial Institution Connectivity Plug-in Interface Definition](#).

To run the plug-in, you must configure and save a format profile that uses the plug-in. The plug-in works with a parser plug-in to enable you to securely connect to a financial institution to retrieve financial data.



Note: There may be cases where a Financial Institution Connectivity Plug-in requires user credentials for a financial institution. NetSuite itself doesn't ask for or store these credentials, but it lets the plug-in save them securely. This can be handled by a third-party service or a customization for the plug-in.

When a Financial Institution Connectivity Plug-in works with a Bank Statement Parser Plug-in, you're able to import bank data for reconciliation. When a Financial Institution Connectivity Plug-in works with a Financial Institution Parser Plug-in, you can import additional data types, including corporate card charges for expense reporting.

For more information about running a Financial Institution Connectivity Plug-in, see [Conditions for Running a Financial Institution Connectivity Plug-in](#). For testing details, see [Conditions for Testing a Financial Institution Connectivity Plug-in](#).

For details about configuring format profiles, see the help topic [Format Profile Creation](#).

A Financial Institution Connectivity Plug-in is also supported by the SuiteCloud Development Framework (SDF). SDF enables you to customize your NetSuite account from an integrated development environment (IDE) on your local computer. For details, see the help topic [SuiteCloud Development Framework](#).

In SDF, you can customize a Financial Institution Connectivity Plug-in by defining `fconnectivityplugin` objects. For details, see the help topic [Financial Institution Connectivity Plug-ins as XML Definitions](#).

For more information about Financial Institution Connectivity Plug-ins, see the following topics:

NetSuite Role	For more information, see ...
All roles	Financial Institution Connectivity Workflows Default Financial Institution Connectivity Plug-in Implementations Capabilities of Using a Financial Institution Connectivity Plug-in Limitations of Using a Financial Institution Connectivity Plug-in

NetSuite Role	For more information, see ...
Developer	Developing a Financial Institution Connectivity Plug-in
Administrator	Administering a Financial Institution Connectivity Plug-in

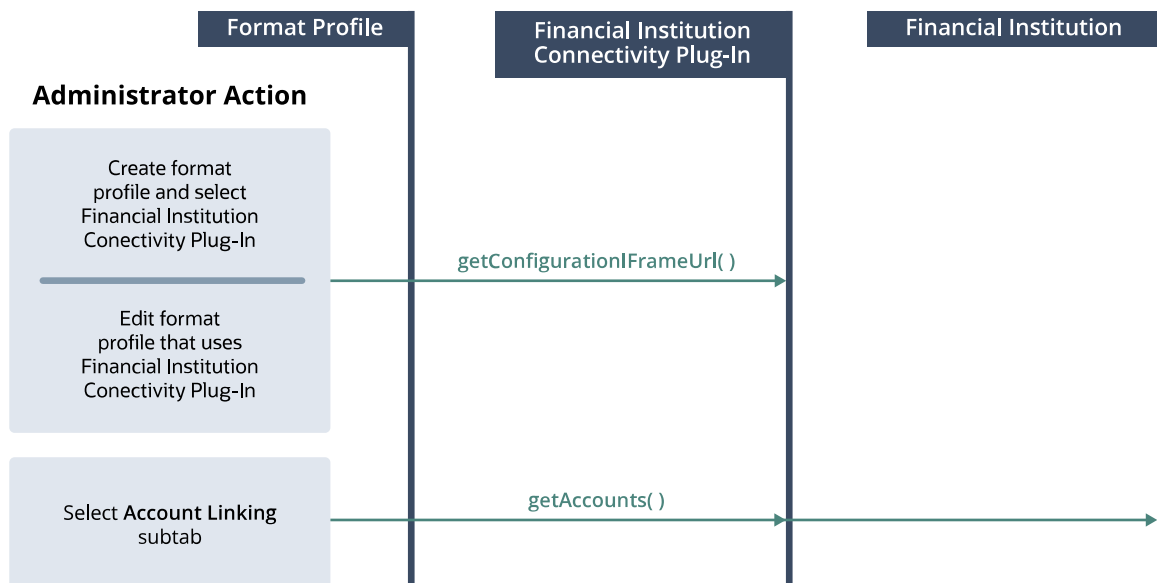
After the administrator performs the necessary one-time setup on a format profile record, the NetSuite user can continue with other stages of the Cash Management workflow. These include transaction matching, reconciliation, and corporate card expense reporting. For details, see the help topics [Bank Data Matching and Reconciliation](#) and [Corporate Card Expenses](#).

Note: This section only covers the development and administration of a Financial Institution Connectivity Plug-in. For details on using a Financial Institution Connectivity Plug-in to import bank data, see the help topic [Automating Imports for Bank Reconciliation](#).

Financial Institution Connectivity Workflows

A Financial Institution Connectivity Plug-in is invoked when you configure a format profile record and during the bank data import process.

The following diagram shows the synchronous workflow for the format profile record:



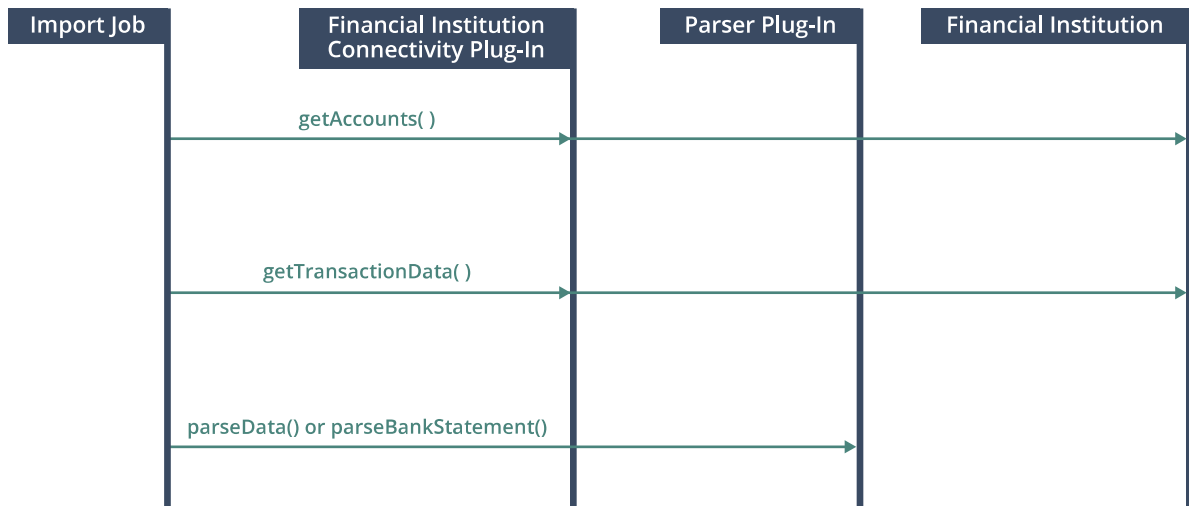
`getConfigurationIFrameUrl()` and `getAccounts()` are interface functions of the Financial Institution Connectivity Plug-in interface, and they do the following:

- [getConfigurationIFrameUrl\(\)](#) – Retrieves a URL for configuring a connection to your financial institution. This function is invoked when you do one of the following:
 - Create a new format profile and select a Financial Institution Connectivity Plug-in from the Connectivity Method dropdown list
 - Edit an existing format profile that uses a Financial Institution Connectivity Plug-in

- **getAccounts()** – Retrieves all available accounts from a financial institution, which enables you to link bank or credit card accounts to corresponding general ledger accounts in NetSuite. This function queries the status of the financial institution accounts, including the date or time since the last update.

Note: **getAccounts()** is only invoked in a format profile with a profile type of Bank Reconciliation.

The following diagram shows the asynchronous workflow for the bank data import process:



getAccounts() and **getTransactionData()** are interface functions of the Financial Institution Connectivity Plug-in interface.

parseData() is an interface function of the Financial Institution Parser Plug-in interface, and **parseBankStatement()** is an interface function of the Bank Statement Parser Plug-in interface.

Note: A Financial Institution Connectivity Plug-in works with a Financial Institution Parser Plug-in or Bank Statement Parser Plug-in. Therefore, a Financial Institution Connectivity Plug-in supports any format that has a Financial Institution Parser Plug-in or Bank Statement Parser Plug-in.

All three functions run when you start a bank data import, and here's what they do:

- **getAccounts()** – Retrieves accounts from your financial institution, which appear on the Account Linking subtab on a Bank Reconciliation format profile record. This subtab is populated with information based on the financial institution accounts for which you have a configured connection. You can link your unlinked accounts to corresponding accounts in NetSuite.

Note: To import data successfully, you'll need at least one active linked account.

- **createAccountData()** – Retrieves your financial data from your financial institution.

If your profile type is Bank Reconciliation, the following takes place:

- When the plug-in is invoked for the first time, NetSuite retrieves the last 60 calendar days of data, by default, or as requested in the `accountRequestsJSON` property. If required, you can modify the date range. For details, see [accountRequestsJSON](#).
- Subsequently, NetSuite only retrieves new data created since the last successful import.

If your profile type is Corporate Card Expenses, NetSuite uses your credentials in the Connectivity Configuration subtab to download charges from the corporate card gateway at each scheduled

import. How much data is downloaded and how it's retrieved depends on the Financial Institution Connectivity Plug-in configuration.

- [parseData\(\)](#) or [parseBankStatement](#) – Consumes the data retrieved by a Financial Institution Connectivity Plug-in and parses it into transactions. To use a connectivity plug-in, you'll need either an existing or new implementation of the Financial Institution Parser Plug-in or Bank Statement Parser Plug-in. To develop these plug-ins, see the help topics [Developing a Financial Institution Parser Plug-in](#) and [Developing a Bank Statement Parser Plug-in](#).

Default Financial Institution Connectivity Plug-in Implementations

In addition to using the Financial Institution Connectivity Plug-in API specifications to create a plug-in, you can use the following default implementations of the plug-in:

- **Bank Feeds Connectivity** – This plug-in is provisioned by the Bank Feeds SuiteApp. Use this plug-in with the Bank Feeds Parser bundled with the SuiteApp, to connect to financial institutions in the United States or Canada, or in Europe. The Bank Feeds SuiteApp enables you to automatically import data daily. For details, see the help topic [Bank Feeds SuiteApp](#).
- **SFTP Connectivity** – This plug-in is provisioned by the Auto Bank Statement Import (ABSI) SuiteApp. Use this plug-in with a Bank Statement Parser Plug-in to connect to a financial institution to retrieve bank statement files, daily. The ABSI SuiteApp uses a Financial Institution Connectivity Plug-in implementation to securely connect to your financial institution's server through Secure File Transfer Protocol connectivity. For details, see the help topic [Auto Bank Statement Import SuiteApp](#).

Capabilities of Using a Financial Institution Connectivity Plug-in

With a Financial Institution Connectivity Plug-in, you can:

- Build and manage your own custom integration to a financial institution API and configure it to fit your needs.

Conversely, the Bank Feeds SuiteApp provides a generic way to connect to a large set of financial institutions. As well, the AMEX Corporate Card Integration SuiteApp enables you to only connect to American Express.

- Connect to financial institutions that are not supported by SuiteApps.

 **Note:** The Bank Feeds SuiteApp is available to connect to financial institutions that operate in:

- Australia
- Asia (Hong Kong, Indonesia, Malaysia, Philippines, Singapore, and Thailand)
- Europe
- Oceania (New Zealand)
- North America (Canada and the United States)

- Retrieve bank data from your financial institution without having to provide user credentials to a third-party service, which the Bank Feeds SuiteApp requires.

In addition to creating plug-ins, the Financial Institution Connectivity interface enables you to develop your own connectivity applications. These provide more options for the way that your system connects to financial institutions.

Limitations of Using a Financial Institution Connectivity Plug-in

When you use a Financial Institution Connectivity Plug-in, keep these limitations in mind:

- The plug-in does not allow you to add IP addresses to an allowlist.
- The plug-in lets you retrieve ZIP files, provided that your Bank Statement Parser Plug-in is capable of handling them.

Developing a Financial Institution Connectivity Plug-in

To develop a Financial Institution Connectivity Plug-in, complete the following steps:

- [Enabling Features for a Financial Institution Connectivity Plug-in](#)
- [Creating a Financial Institution Connectivity Plug-in Script File](#)

Enabling Features for a Financial Institution Connectivity Plug-in

To begin development of a Financial Institution Connectivity Plug-in, make sure that the Server SuiteScript and Client SuiteScript features are enabled on the development account.

To enable features for the plug-in:

1. Choose Setup > Company > Enable Features.
2. On the **SuiteCloud** subtab, make sure **Server SuiteScript** and **Client SuiteScript** are checked. If necessary, check each box and agree to the Terms of Service.
3. Click **Save**.

After you enable the Server SuiteScript and Client SuiteScript features, you can create a plug-in script file. See [Creating a Financial Institution Connectivity Plug-in Script File](#).

To install and set up your plug-in, see [Administering a Financial Institution Connectivity Plug-in](#).

Creating a Financial Institution Connectivity Plug-in Script File

You must implement each Financial Institution Connectivity Plug-in interface function in a JavaScript file (with a .js extension) to define the behavior of the plug-in implementation. You can use the SuiteCloud IDE, another JavaScript IDE, or any text editor to create your plug-in script file.

The following sample implements a Financial Institution Connectivity Plug-in that returns static data using the N/search SuiteScript module in the plug-in implementation script file:

Note: For details on the N/search SuiteScript module, see the help topic [N/search Module](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType fiConnectivityPlugin
4   * @NModuleScope SameAccount
5   */
6  define(['N/search'],
7    function (search)
8    {
9
10     // internal function used to load configuration for this plug-in from a custom record
11     function loadConfiguration(configurationId)
12     {
13       var searchResults = search.create({
14         type: 'customrecord_sampleconfig',
15         filters: [{
16           name: 'custrecord_configurationid',
17           operator: 'is',
18           values: [configurationId]
19         }]
20       });
21       return searchResults.run().getRange({start: 0, end: 1});
22     }
23
24     function getConfigurationIFrameUrl(context)
25     {
26       var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
27       context.configurationIFrameUrl = "/app/site/hosting/scriptlet.nl?script=1&deploy=1&configurationId=" + configurationId;
28     }
29
30     function getAccounts(context)
31     {
32       var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
33       var configuration = loadConfiguration(configurationId);
34       context.addAccount({
35         accountMappingKey: "12345",
36         displayName: "Checking (XXXX11)",
37         accountType: "BANK",
38         currency: "USD",
39         groupName: "Bank of America",
40         lastUpdated: "2020-06-30T01:23:45"
41       });
42     }
43
44     function getTransactionData(context)
45     {
46       var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
47       var configuration = loadConfiguration(configurationId);
48       var accountRequests = JSON.parse(context.accountRequestsJSON);
49       if (accountRequests != null) {
50         accountRequests.forEach(function (accountRequest) {
51           var accountId = accountRequest.accountMappingKey;
52           var fromDateTime = accountRequest.dataStartTime;
53           var toDateTime = accountRequest.dataEndTime;
54
55           var downloadedData = "... Retrieve the account data using a web service request or a file transfer...";
56
57           context.addDataChunk({dataChunk: downloadedData});
58         });
59       }
60       context.returnAccountRequestsJSON({accountsJson: context.accountRequestsJSON});
61     }
62
63     return {
64       getConfigurationIFrameUrl: getConfigurationIFrameUrl,
65       getAccounts: getAccounts,
66       getTransactionData: getTransactionData

```

```

67 |     }
68 | });

```

For details and breakdowns of the sample, see [Financial Institution Connectivity Plug-in Interface Definition](#).

Guidelines for Creating a Plug-in Implementation Script File

Use the following guidelines when creating the plug-in implementation script file:

- You can name the plug-in script file anything you want, as long as it includes all required interface functions.
- You can include utility files with helper functions when you build the plug-in implementation. See [Adding the Financial Institution Connectivity Plug-in Implementation](#).
- Because of how strings work in NetSuite, you can't use the JavaScript case statement in the plug-in script file.
- Each interface function allows up to 1000 usage units.

Administering a Financial Institution Connectivity Plug-in

To install and set up a Financial Institution Connectivity Plug-in, complete the following steps:

- [Enabling Features for a Financial Institution Connectivity Plug-in](#)
- [Adding the Financial Institution Connectivity Plug-in Implementation](#)
- [Configuration for a Financial Institution Connectivity Plug-in](#)
- [Conditions for Running a Financial Institution Connectivity Plug-in](#)
- [Conditions for Testing a Financial Institution Connectivity Plug-in](#)

Enabling Features for a Financial Institution Connectivity Plug-in

See [Enabling Features for a Financial Institution Connectivity Plug-in](#).

Adding the Financial Institution Connectivity Plug-in Implementation

After creating the plug-in script file, upload and implement the file in NetSuite.

To add the plug-in implementation:

1. In NetSuite, go to Customization > Plug-ins > Plug-in Implementations > New.
NetSuite displays the Upload Plug-in Implementation page.
2. For the **Script File**, click the plus icon.
3. Click **Choose File**, then select the JavaScript file.
4. Click **Save**.
5. Click **Create Plug-in Implementation**.

6. On the Plug-in Implementation page, enter the following information:

 **Note:** You don't need to select the plug-in type or implementation. Since the plug-in uses SuiteScript 2.0, NetSuite pulls this info from the script file.

Option	Description
Name	User-friendly name for the implementation. This name appears in the plug-in implementation list.
ID	Internal ID for the implementation for use in scripting. If you do not provide an ID, NetSuite will provide one for you after you click Save . As a developer, when you write code that uses this implementation, you will reference the implementation using this ID.
Status	Current status for the implementation. Choose Testing to have the implementation accessible to the owner of the implementation. Choose Released to have the implementation accessible in a production environment
Log Level	Logging level you want for the script. Select Debug, Audit, Error, or Emergency. The log information appears on the Execution Log subtab for the implementation after you create it. Go to Customization > Plug-ins > Plug-in Implementations, select your implementation, and then click the Execution Log subtab.
Description	Optional description of the implementation.
Owner	User account that owns the implementation. Default is the name of the current user

7. On the **Unhandled Errors** subtab, select which individual or individuals are notified if script errors occur. By default the **Notify Script Owner** box is checked.

 **Tip:** To enter multiple email addresses in the **Notify Emails** box, separate email addresses with a semi-colon.

8. Click **Save**.

Configuration for a Financial Institution Connectivity Plug-in

You'll do all configuration for a Financial Institution Connectivity Plug-in on the Format Profile page. The configuration steps required depend on the plug-in's initial implementation.

For details on configuring connectivity to a bank or credit card company, see the help topic [Configuring a Connection to a Financial Institution \(Without a SuiteApp\)](#).

For details on configuring connectivity to a corporate card company, see the help topic [Configuring a Connection to American Express](#).

Conditions for Running a Financial Institution Connectivity Plug-in

To run a Financial Institution Connectivity Plug-in, you must create a format profile that uses the plug-in with a Financial Institution Parser Plug-in or Bank Statement Parser Plug-in.

A Financial Institution Connectivity Plug-in works with a Bank Statement Parser Plug-in to enable you to automatically retrieve data for bank reconciliation. For details about automating imports of bank or credit card data for reconciliation, see the help topic [Automating Imports for Bank Reconciliation](#).

When a Financial Institution Connectivity Plug-in works with a Financial Institution Parser Plug-in, you can retrieve additional data types, including corporate card charges for expense reporting. For details about automating imports of corporate card data for expense reporting, see the help topic [Automating Imports for Expense Reporting](#).

To use the default Financial Institution Connectivity Plug-in implementation available through the Bank Feeds SuiteApp, see the help topic [Bank Feeds SuiteApp](#).

Note: To import data successfully, you'll need at least one active account mapping on the format profile record.

When you save a format profile configured for automatic import for the first time:

- NetSuite creates a daily import schedule to retrieve financial data. See the help topic [Automatic Bank Data Import](#).
- If you're importing data for bank or credit card reconciliation, the Financial Institution Connectivity Plug-in runs. NetSuite then connects to your financial institution to get the last 60 days of data from your linked accounts.

After you save the format profile for the first time, later imports can happen in these ways:

- During a scheduled import of bank or credit card data, or corporate card expense data.
- If you're importing data for bank or credit card reconciliation:
 - When you edit and save an existing format profile configured for connectivity to a financial institution (see the help topic [Format Profile Creation](#)). The Financial Institution Connectivity Plug-in runs, enabling NetSuite to connect to a financial institution to retrieve new data from existing accounts since the last successful import. For new accounts, NetSuite imports the last 60 calendar days of data.

Note: There are cases where an import is not initiated. For example, NetSuite cannot import data from the same format profile twice within a 30-minute period.

- When you initiate a manual update of imported data for an account on the Match Bank Data page. The Financial Institution Connectivity Plug-in runs, enabling NetSuite to import new data from the account since the last successful import. Imported account data appears in the Imported Bank Data grid. For details, see the help topic [Updating Imported Bank Data](#).

Note: You should only use this method in these situations:

- You have a need to test a plug-in for a single financial institution account.
- The selected account does not contain the latest data.
- There was an import error.

To verify that the plug-in is working, see [Conditions for Testing a Financial Institution Connectivity Plug-in](#).

Conditions for Testing a Financial Institution Connectivity Plug-in

To test a Financial Institution Connectivity Plug-in, run the plug-in to import bank data, then check your import.

To run the plug-in, see [Conditions for Running a Financial Institution Connectivity Plug-in](#).

If you get an import error, check the plug-in's execution log for details. Go to Customization > Plug-ins > Plug-in Implementations. Click the View link of the Financial Institution Connectivity Plug-in, and then on the Plug-in Implementation page, click the Execution Log subtab.

To view your imported data, see the help topic [Viewing Imported Banking Data](#).

Financial Institution Connectivity Plug-in Interface Definition

The Financial Institution Connectivity Plug-in interface includes the following interface functions and containing interface objects and object functions:

Interface Function	Containing Interface Objects	Containing Object Functions
getConfigurationIFrameUrl()	pluginConfiguration	getConfigurationFieldValue() configurationIFrameUrl
getAccounts()	pluginConfiguration	getConfigurationFieldValue() addAccount()
getTransactionData()	pluginConfiguration	getConfigurationFieldValue() accountRequestsJSON addDataChunk() returnAccountRequestsJSON()

getConfigurationIFrameUrl()

Function Declaration	<code>function getConfigurationIFrameUrl(context)</code>
Type	Interface function
Description	Provides the user interface for configuring a connection to your financial institution. For most plug-in authors, this is a URL to a Suitelet. This URL appears on the Connectivity Configuration subtab on a format profile record.
Returns	void
Input Parameters	context – a JavaScript object with the following properties: ■ pluginConfiguration ■ configurationIFrameUrl

Example

```

1 function getConfigurationIFrameUrl(context)
2 {
3     var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
4     context.configurationIFrameUrl = "/app/site/hosting/scriptlet.nl?script=1&deploy=1&configurationId=" + configurationId;
5 }

```

pluginConfiguration

Type	Interface object
-------------	------------------

Description This object allows the plug-in to retrieve user-supplied configuration data (field values) for this plug-in.



Note: If you are developing your own plug-in, use this interface function to access the configuration_id value.

Object Functions [getConfigurationFieldValue\(\)](#)

Parent Object(s) [getConfigurationIFrameUrl\(\)](#)

[getAccounts\(\)](#)

[getTransactionData\(\)](#)

Child Object(s) N/A

getConfigurationFieldValue()

Function Declaration `function getConfigurationFieldValue(options)`

Type Object function

Description This is an object function of [pluginConfiguration](#) that allows the plug-in to retrieve a named configuration value.

For all Financial Institution Connectivity Plug-ins that you develop, the configuration_id value is available. This is a unique identifier at the format profile level. It can be used to refer to a format profile, even if the record is not created.

Returns string

Input Parameters options – a JavaScript object with the property fieldName, which is a string. This property is required and refers to the name of the configuration value to retrieve.

Example

```
1 | context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
```

configurationIFrameUrl

Type string

Description The URL to a user interface for configuring a connection to your financial institution. This URL appears on the Connectivity Configuration subtab on the Format Profile page.

Example

```
1 | var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
2 | context.configurationIFrameUrl = "/app/site/hosting/scriptlet.nl?script=1&deploy=1&configurationId=" + configurationId;
```

getAccounts()

Function Declaration	function getAccounts(context)
Type	Interface function
Description	Retrieves all available accounts from a financial institution. This function enables you to link bank or credit card accounts to corresponding general ledger accounts in NetSuite. The function is invoked when you open the Account Linking subtab on a Bank Reconciliation format profile record, or when bank data is imported into NetSuite. To add a financial institution account, see addAccount() .
Returns	void
Input Parameters	context – a JavaScript object with the following properties: ■ pluginConfiguration ■ addAccount() ■ isRetryAllowed() ■ retry()

Example

```

1 function getAccounts(context)
2 {
3     var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
4     var configuration = loadConfiguration(configurationId)
5     context.addAccount({
6         accountMappingKey: "12345",
7         displayName: "Checking (XXX11)",
8         accountType: "BANK",
9         currency: "USD",
10        groupName: "Bank of America",
11        lastUpdated: "2020-06-30T01:23:45"
12    });
13 }

```

If you want the plug-in to initiate a bank data import retry, make sure `isRetryAllowed()` is true and then call `retry()`.

```

1 if (context.isRetryAllowed()) {
2     context.retry({deltaMinutesLater: 30, currentFailureReason: "Timed out"});
3 }

```

Reasons why you may want to initiate an import retry can include the following:

- Your financial institution goes down.
- There is too much data.
- There was a time-out for the import.

An import retry is initiated during the import process. `retry()` causes the current import job to be marked as failed, and then initiates a retry.

pluginConfiguration

See [pluginConfiguration](#).

getConfigurationFieldValue

See [getConfigurationFieldValue\(\)](#).

addAccount()

Function Declaration `function addAccount(options)`

Type Object function

Description This function enables the plug-in to add bank or credit card accounts you want to link to corresponding accounts in NetSuite. NetSuite displays the supplied accounts on the Account Linking subtab on a Bank Reconciliation format profile record.

This function should be called for each account being linked.

Returns void

options is a JavaScript object with the following input parameters:

Parameter	Type	Required	Description	Since
accountMappingKey	String	Yes	The financial institution account's unique identifier. For compliance reasons, this cannot be a credit card number. To ensure that the account mapping key does not contain a credit card number, NetSuite removes all non-numeric characters. ■ If the remaining string is less than 13 characters or greater than 20 characters, NetSuite accepts the account mapping key. ■ If the remaining string is greater than 12 characters and less than 21 characters, NetSuite performs a Luhn check. If the check passes, NetSuite blocks you from using the account mapping key. If the check fails, NetSuite accepts the account mapping key.	2020.1
displayName	String	No	The financial institution account name capable of being displayed.	2020.1
accountType	String	Yes	The account type that should map to a NetSuite account type. Valid types include: ■ ACCOUNTS_PAYABLE ■ ACCOUNTS_RECEIVABLE ■ BANK ■ COGS ■ CREDIT_CARD ■ DEFERRED_EXPENSE ■ DEFERRED_REVENUE ■ EQUITY ■ EXPENSE ■ FIXED_ASSET ■ INCOME ■ LONG_TERM_LIABILITY	2020.1

Parameter	Type	Required	Description	Since
			■ NON_POSTING ■ OTHER_ASSET ■ OTHER_CURRENT_ASSET ■ OTHER_CURRENT LIABILITY ■ OTHER_EXPENSE ■ OTHER_INCOME ■ STATISTICAL ■ UNBILLED_RECEIVABLES	
currency	String	Yes	The financial institution account currency code. Use this on the Account Linking or Employee Linking subtab to filter by NetSuite accounts with the same currency. The currency input parameter must be a valid ISO 4217 currency code.	2020.1
groupName	String	Yes	The financial institution name that can be displayed, which is used for grouping accounts from the same financial institution.	2020.1
lastUpdated	String	No	The last time the financial institution updated data for an account. If the financial institution has not updated the account since the last import, then you know there is no new data to import. If getAccounts() returns a lastUpdated value for an account, NetSuite calls getTransactionData(), and the dataEndTime field for that account is populated based on the lastUpdated value. This indicates to the plug-in that NetSuite is requesting bank data up to the date provided by the lastUpdated value.	2020.1

Example

```

1 context.addAccount({
2   accountMappingKey: "12345",
3   displayName: "TD Bank - 98765",
4   accountType: "BANK",
5   currency: "USD",
6   groupName: "TD",
7   lastUpdated: '2018-01-01T15:00:00'
8 })

```

isRetryAllowed()

Function Declaration function isRetryAllowed()

Type Object function

Description This function is invoked by the plug-in to check whether a bank import retry is allowed.

- A retry is not permitted if it is not called during an import job.
- If a retry is called during an import job, a retry is permitted if there has been no more than one failed import job in the past two hours for a format profile or for a single financial institution account.

Returns boolean

Input Parameters N/A

Example

```
1 | if (context.isRetryAllowed()) {
2 |     context.retry({deltaMinutesLater: 30, currentFailureReason: "Timed out"});
3 | }
```

retry()

Function Declaration function retry(options)

Type Object function

Description This method is invoked by the plug-in to schedule a bank import retry deltaMinutesLater minutes in the future. The current import job will fail and a failure reason will appear. If your role has the Import Online Banking File permission with create-level access at a minimum, the failure reason will include an error from the plug-in. Otherwise, the failure reason will be more generic.

The method may throw an exception with one of the following error codes:

- SSS_RETRY_NOT_ALLOWED: retry is called when retry is not allowed
- SSS_INVALID_RETRY_DELAY: deltaMinutesLater is outside of valid range
- SSS_RETRY_FAILURE_REASON_SIZE_LIMIT_EXCEEDED: currentFailureReason is too long

Returns void

Parameter	Type	Required	Description	Since
deltaMinutesLater	Number	Yes	How many minutes into the future to schedule the import retry. Must be greater than or equal to 30, and less than or equal to 240.	2020.2
currentFailureReason	String	No	The reason an import retry is needed.	2020.2

Example

```
1 | if (context.isRetryAllowed()) {
2 |     context.retry({deltaMinutesLater: 30, currentFailureReason: "Timed out"});
3 | }
```

getTransactionData()

Function Declaration function getTransactionData(context)

Type Interface function

Description This function enables the plug-in to invoke a Financial Institution Parser Plug-in or Bank Statement Parser Plug-in to parse content into transactions. This happens when a bank data import is initiated.



Note: For a successful import of bank or credit card data, you need at least one active linked account on the format profile record.

Returns void

Input Parameters context – a JavaScript object with the following properties:

- [pluginConfiguration](#)
- [accountRequestsJSON](#)
- [addDataChunk\(\)](#)
- [returnAccountRequestsJSON\(\)](#)
- [isRetryAllowed\(\)](#)
- [retry\(\)](#)

Example

```

1 function getTransactionData(context)
2 {
3     var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
4     var configuration = loadConfiguration(configurationId);
5     var accountRequests = JSON.parse(context.accountRequestsJSON);
6     if (accountRequests != null) {
7         accountRequests.forEach(function (accountRequest) {
8             var accountId = accountRequest.accountMappingKey;
9             var fromDateTime = accountRequest.dataStartTime;
10            var toDateTime = accountRequest.dataEndTime;
11
12            var downloadedData = "... Retrieve the account data using a web service request or a file transfer...";
13
14            context.addDataChunk({dataChunk: downloadedData});
15        });
16    }
17    context.returnAccountRequestsJSON({accountsJson: context.accountRequestsJSON});
18 }

```

If you want the plug-in to initiate a bank data import retry, make sure `isRetryAllowed()` is true and then call `retry()`.

```

1 if (context.isRetryAllowed()) {
2     context.retry({deltaMinutesLater: 30, currentFailureReason: "Timed out"});
3 }

```

Reasons why you may want to initiate an import retry can include the following:

- Your financial institution goes down.
- There is too much data.
- There was a time-out for the import.

An import retry is initiated during the import process. `retry()` causes the current import job to be marked as failed, and then initiates a retry.

pluginConfiguration

See [pluginConfiguration](#).

getConfigurationFieldValue

See [getConfigurationFieldValue\(\)](#).

accountRequestsJSON

Type string

Description A list of financial institution accounts required for the plug-in to query bank data, as well as the required data date ranges. The information retrieved is provided as a JSON format string:

Note: The accountRequestsJSON property is only applicable to Bank Reconciliation format profile setups.

```
1 [{\"dataStartTime\":\"2019-08-20T02:30:11\", \"dataEndTime\":\"2019-09-20T02:30:11\", \"accountMappingKey\":\"acct-123\"},
2 * {\"dataStartTime\":\"2019-08-20T02:30:11\", \"dataEndTime\":\"2019-09-20T02:30:11\", \"accountMappingKey\":\"acct-456\"}]
```

The properties embedded in the JSON string include the following:

- dataStartTime - The requested start time of the data to be retrieved. If the Bank Reconciliation format profile is saved for the first time, NetSuite downloads data up to 60 calendar days old. After the first import, dataStartTime can also be dependent on the last successful import.
- dataEndTime - The requested end time of the data to be retrieved. dataEndTime is the time when a bank data import is initiated.
- accountMappingKey - The financial institution account's unique identifier.

For an example of where the property accountRequestsJSON is used, see [getTransactionData\(\)](#).

addDataChunk()

Function Declaration function addDataChunk(options)

Type Object function

Description This function enables you to transmit the financial institution's data file to NetSuite as a series of chunks. It encrypts the incoming chunks before storing them in the database. Each chunk size has a 25-million character limit.

Returns void

Input Parameters options – a JavaScript object with the property dataChunk, which is a chunk of data from your financial institution's data file. dataChunk is required and a string.

Example

```
1 | context.addDataChunk({dataChunk: "...transaction data..." });
```

returnAccountRequestsJSON()

Function Declaration function returnAccountRequestsJSON(options)

Type	Object function
Description	This function enables you to return the list of accounts queried by the plug-in, as well as the reason for any query failures. The import job fails when one or more accounts return a failure reason during the import. Failure reasons are displayed on the Banking Import History page.
	 Note: This function is only applicable to Bank Reconciliation format profile setups. If an account is provided by accountRequestsJSON but not returned by the Financial Institution Connectivity Plug-in interface, it is assumed that there is no account-specific failure.
Returns	void
Input Parameters	options – a JavaScript object with the property accountsJson. accountsJson contains the list of financial institution accounts queried by the plug-in, as well as the reason for any query failures. This information is returned as a JSON format string:

```

1 [{\"dataStartTime\": \"2019-08-20T02:30:11\", \"dataEndTime\": \"2019-09-20T02:30:11\", \"accountMappingKey\":
  \"acct-123\"},
2 * {\"dataStartTime\": \"2019-08-20T02:30:11\", \"dataEndTime\": \"2019-09-20T02:30:11\", \"accountMappingKey\":
  \"acct-456\", \"failureReason\": \"Invalid account\"}]

```

The properties embedded in the JSON string include the following:

- dataStartTime - The requested start time of the data to be retrieved. If a Bank Reconciliation format profile is saved for the first time, NetSuite downloads data up to 60 calendar days old. After the first import, dataStartTime can also be dependent on the last successful import.
- dataEndTime - The requested end time of the data to be retrieved. dataEndTime is the time when a bank data import is initiated.
- accountMappingKey - The financial institution account's unique identifier.
- failureReason - If provided, indicates the reason for the failed imported statement.

isRetryAllowed()

See [isRetryAllowed\(\)](#).

retry()

See [retry\(\)](#).

Error Propagation

You should raise all user-facing error messages using the N/error SuiteScript module. For details on this module, see the help topic [N/error Module](#).

In all user-facing errors, you must set the name property to FICONNECTIVITY_USER_ERROR. Errors not raised with the FICONNECTIVITY_USER_ERROR name property, as well as any exception not handled, are not surfaced to users. Instead, NetSuite displays a generic error message on the Format Profile page. Details about the error are available in the plug-in's execution log.

The following sample code illustrates how to raise a user-facing error:

```

1 throw error.create({ // error variable is the imported 'N/error' module
2   name: 'FICONNECTIVITY_USER_ERROR',
3   message: 'This message will be displayed on the Format Profile page'

```



```
4 | });
```

The next sample code illustrates how to raise a non user-facing error:

```
1 | throw 'This message will not be displayed on the Format Profile page';
```